Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 35

Graphs

Shortest-Path in Weighted Graphs

Recall: BFS and Shortest-Path

- An important application in a graph: Given a pair of source-target nodes (s,t), check if t reachable from s, and if so, find the distance d(s,t)
  - Run BFS from source s and if t is reached, report the level k s.t. $t \in L_{s,k}$
- Can we also find a shortest path from s to t?
  - Yes! When a node v is discovered from u, record `discovered[v]=u` (instead of just a boolean flag); use, say, `-1` to indicate not discovered, and let `discovered[s]=s`
  - Let $u_0 = t$, $u_{i+1} =$ `discovered[`u_i`]`, until $u_k == s$. Path is $u_k, u_{k-1}, \ldots, u_0$
  - In the traversal template, with any container , gives some s-t path

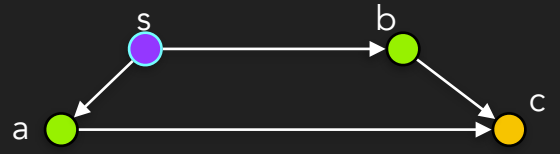
Weighted Graph

- We had defined d(s,v) = minimum number of edges in a path from s to v
  - Each edge contributes equally to the length of a path
- But in many applications, not all edges are equal
  - Cost of a step, e.g., distance on a map, latency in a network link, unlikelihood in a probabilistic model, ...
  - If the costs are all small integers, can model an edge with cost k by a path of length k (involving k-1 dummy nodes). But not feasible/efficient for arbitrary costs.
- Weighted Graph: A graph with non-negative costs, or weights, on the edges. Now, path-length adds up edge weights. d(s,v) = minimum length of a path from s to v.

Shortest Path in a Weighted Graph

Dijkstra's Algorithm

- As in BFS, we will explore the nodes nearest to the source first
- Main difference: in BFS we find the distance to a node when we discover it. In exploring a weighted graph this is not true
  - E.g., c discovered through path s→a→c since a is closest to s, but s→b→c can be shorter!



- Idea: Allow distance to be updated each time a node is discovered
  - This will be enough to ensure that we know the next nearest node to explore at any point!

Attendance Break!

Login using CSE LDAP id at

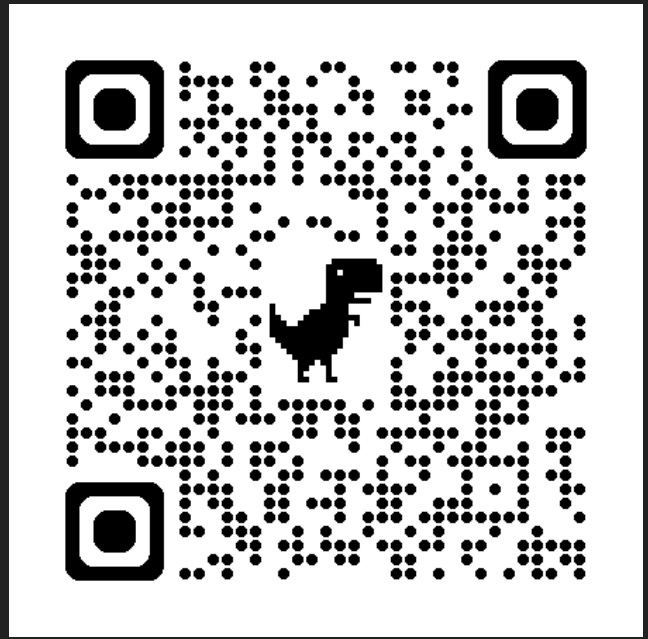
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



Dijkstra's Algorithm

- We shall use a "priority queue" as the frontier
  - holds (u,k) pairs (k possibly ∞); we call k = D(u), the recorded distance of u.
  - (u,k) can be updated to (u,k') only if k' < k
  - extraction yields the node with the minimum recorded distance

```
distance(s,t) {
  for (v : V) frontier.insert(v,∞);
  frontier.decrease(s,0);
  while(true) {
    (u,k) = frontier.extract_min();   // guaranteed that d(s,u) = k
    if(u==t) return k;
    if(k==∞) return ∞;                // all remaining nodes unreachable
    for ( v: graph.neighbours(u) ) {
      duv = k + graph.weight(u,v);    // distance from s to v using (u,v)
      frontier.decrease(v,duv);       // updates (v,j) → (v,duv) iff duv < j
    }
  }
}
```

If k == ∞, then D(w) == ∞ for all unexplored w. So no edge from any explored node to unexplored nodes: i.e., a *cut* between explored and unexplored

Dijkstra's Algorithm

- Claim: When extracting any reachable node u, recorded distance D(u) = d(s,u)
- Base case: first extracted item is (s,0) and d(s,s)=0.
- Induction: S be the set of nodes extracted so far, and T = V \ S.
  IH: ∀u∈S, D(u) = d(s,u). Next extracted node be w ∈ T (assumed reachable).
  - Consider a shortest s-w path P, and v be the first node in T on P
    Then P upto v is a shortest s-v path (else can find a shorter walk, and hence path, to w) and d(s,v) ≤ d(s,w) ≤ D(w) ≤ D(v).
  - Let s ⇢ u → v be this path. By choice of v, we have u ∉ T. i.e., u∈S. Hence, by IH, when u was extracted, D(u) = d(s,u) and we updated D(v) = D(u) + w(u,v) = d(s,v). (D(v) cannot increase after that.)
  - So d(s,v) ≤ d(s,w) ≤ D(w) ≤ D(v) ≤ d(s,v). Hence D(w) = d(s,w)

D(w) is the length of an s-w path we have seen

non-negative edge weights

Dijkstra's Algorithm

- How do we implement the priority queue?
- Practical choice: A binary (min-)heap

Can be made more efficient asymptotically, using "Fibonacci heaps," but rarely used in practice

  - Additional feature needed: locate an entry for a node, and decreasing its key
    - Once node located on the heap, decreasing a key can be done in O(log |V|) time, by bubbling up
    - How do you locate a node on the heap?
      - Use a directory (vector) to map each node to a position in the heap
        - While bubbling up, the directory should be updated
  - Alternative: Instead of decreasing a key, just and a new entry with the new key. On extracting, check if the same node was extracted earlier, and if so ignore it and continue. Slightly more expensive (|E| items stored instead of |V|), but can use std::priority_queue as a blackbox

Dijkstra's Algorithm: Time

- Initialisation of priority queue with (u,∞) for all u
  - Can actually be omitted by just not inserting them into the heap, and treating items not in the heap as having key ∞
- Every reachable node is extracted exactly once: $O(|V_{rch}|)$ iterations
  - In each iteration: extract an element from the priority queue with at most $O(|V_{rch}|)$ items, in $O(\log |V_{rch}|)$ time.
- Inner loop, across all iterations of the outer loop, explores reachable edges: $|E_{rch}|$ iterations, and in each iteration decrease a key: $O(|E_{rch}| \log |V_{rch}|)$ time
- Total time: $O(|E_{rch}| \log |V_{rch}|)$ since $|V_{rch}| \leq |E_{rch}| + 1$

With Fibonacci heaps decrease-key is O(1) amortised time: $O(|E_{rch}| + |V_{rch}| \log |V_{rch}|)$

Dijkstra's Algorithm: Variations

- We proved that for every node u, when it is extracted, recorded distance D(u) equals the distance d(s,u)
  - If we don't use a specific target t, Dijkstra computes d(s,u) for all u (with the same time complexity): Single-Source Shortest Path (SSSP)
- We can extract the actual shortest paths by recording not just D(u) but also the predecessor from which D(u) was realised
  - In SSSP, this gives a shortest-paths tree, with s as root
- In many applications (like routing in a road network), additional information beyond the graph topology is available to guide the search
  - A* algorithm is a variation of Dijkstra (single target version) where the priority queue uses key(u) = D(u) + estimate(u,t)
- Negative edge weights? Shortest path not well-defined if negative cycles reachable.
  - But even if no negative cycles, Dijkstra doesn't work. (Slower "Bellman-Ford" works.)